

APIx — Test Stack & Test Plan

Team CodeCrew | Smart India Hackathon 2026 | PS ID: SIH26056

1. Testing Philosophy

APIx separates two fundamentally different kinds of "correctness" and tests them separately:

1. **Code correctness** — does each module do what it's supposed to do, mechanically? (dedup actually dedups, the index formula computes what the formula says, the API returns the right schema.) This is covered by the automated pytest suite.
2. **Real-world accuracy** — is the resulting index actually a good measure of Indian airfares? This can't be unit-tested in the traditional sense; it's covered by the **econometric backtest** against DGCA's independently published benchmark data, treated as its own validation layer rather than a unit test.

A change can pass every unit test and still produce an economically meaningless index (e.g., wrong weights) — which is exactly why the backtest is a first-class, separately reported part of the test stack, not an afterthought.

2. Test Tooling

Tool	Role
pytest ($\geq 8.0.0$)	Test runner and framework for all automated tests
pytest fixtures (<code>client</code> fixture in <code>test_api.py</code>)	Reusable FastAPI <code>TestClient</code> setup for endpoint tests
scipy.stats.pearsonr	Correlation computation for backtest validation
NumPy	RMSE/MAPE computation for backtest validation
FastAPI TestClient	In-process HTTP testing of the API layer without a live server
Docker / docker-compose	Environment reproducibility so tests run identically across machines

Dependency versions are pinned/floored in `requirements.txt` (fastapi $\geq 0.110.0$, pydantic $\geq 2.5.0$, streamlit $\leq 1.51.0$, pandas $\geq 2.0.0$, numpy $\geq 1.24.0$, scipy $\geq 1.11.0$, pytest $\geq 8.0.0$), so the test environment is reproducible via `pip install -r requirements.txt`.

Run command: `python -m pytest tests/ -v`

3. Test Levels

Level	What it covers	Where
Unit tests	Individual functions/classes in isolation (cleaning logic, index formula math, watchdog tier classification)	<code>test_cleaner.py</code> , <code>test_index_engine.py</code> , <code>test_watchdog.py</code>
Integration tests	Components working together through a realistic call path (API endpoints hitting the real service stack)	<code>test_api.py</code>
Data/structure validation tests	Config and schema shape checks (PSD weight structure, DGCA benchmark structure, export schema)	<code>test_esankhyiki.py</code> , <code>test_backtest.py</code>
Econometric validation (not pytest)	Whether the computed index is empirically accurate against an independent real-world benchmark	<code>backtester.py</code> $\rightarrow$ <code>evaluate_90day_backtest()</code>
Manual/exploratory testing	Dashboard usability, visual sanity-checking of charts, live demo walkthroughs	Streamlit dashboard, run ad hoc

4. Automated Test Suite — Full Inventory

27 test functions across 7 modules, all currently passing.

`tests/test_scrapers.py` — Ethical Scraping Engine (4 tests)

Test	What it verifies
<code>test_ethical_guard_robots_txt</code>	A scrape request is correctly blocked/allowed based on robots.txt rules — the core ethical-compliance guarantee
	Requests carry proper, non-deceptive headers

<code>test_ethical_headers</code>	(realistic user-agent rotation, no spoofed identity)
<code>test_dynamic_pricing_decay</code>	The synthetic RM engine's fare curve behaves correctly as departure date approaches — specifically asserts the T+1 fare is at least 1.70x the T+45 fare for the same route, quantifying the expected revenue-management decay rather than just checking direction
<code>test_generate_daily_quotes_coverage</code>	The synthetic engine generates quotes covering all 30 basket corridors and all 5 advance-purchase windows with no silent gaps (does not test the 3-epoch intraday sampling weighting — see Section 9, gap #7)

`tests/test_cleaner.py` — Cleaning & Fare Decomposition (4 tests)

Test	What it verifies
<code>test_fare_decomposition_identity</code>	Base fare + GST + UDF + PSF + convenience fee sums back exactly to the total fare (no leakage/double-counting in the decomposition)
<code>test_ota_convenience_fee_captured</code>	OTA-specific convenience fees are correctly extracted and attributed per platform
<code>test_data_cleaner_deduplication</code>	Duplicate quotes are correctly identified and collapsed, with direct-airline sources prioritized over OTA re-listings
<code>test_sold_out_not_zero_filled</code>	A sold-out flight is never recorded as a ₹0 fare — confirms the imputation path is actually used instead of a naive default

`tests/test_index_engine.py` — Two-Tier Jevons-Laspeyres Index (4 tests)

Test	What it verifies
<code>test_two_tier_identity</code>	If all input prices are identical, the index returns exactly 100 (no base-period drift) — a basic correctness invariant of any price index
<code>test_two_tier_proportionality</code>	A uniform X% price increase across all inputs produces an index reading proportionally consistent with X% (the index isn't dampening or amplifying incorrectly)

<code>test_two_tier_dgca_corridor_weighting</code>	Corridor-level aggregation actually uses DGCA traffic weights, not an equal-weighted average — the core economic correctness check for Tier 2
<code>test_two_tier_surge_damping_property</code>	The Jevons geometric mean at the elementary level correctly dampens the influence of an extreme single-fare spike relative to what an arithmetic mean would do — validates the <i>reason</i> Jevons was chosen at Tier 1

`tests/test_backtest.py` — DGCA Benchmark Validation (2 tests)

Test	What it verifies
<code>test_dgca_90day_backtest_performance</code>	The end-to-end backtest runs and produces <code>r</code> /MAPE/RMSE values that clear the documented acceptance thresholds ($r \geq 0.88$, $\text{MAPE} < 5.0\%$, $\text{RMSE} < ₹450$)
<code>test_dgca_monthly_benchmarks_structure</code>	The hardcoded DGCA benchmark dataset has the expected shape (3 months $\times$ 6 named routes + weighted average) before it's used for comparison

`tests/test_api.py` — REST API Integration (6 tests)

Test	What it verifies
<code>test_root_endpoint</code>	<code>/</code> returns <code>status: OPERATIONAL</code> and confirms "NSO (MoSPI)" is listed among the declared agency consumers
<code>test_health_endpoint</code>	<code>/api/v1/health</code> returns <code>status: HEALTHY</code> and a non-zero <code>daily_index_records</code> count — the basic liveness contract any consuming system would check first
<code>test_latest_index_endpoint</code>	<code>/api/v1/index/latest</code> returns all five index fields (<code>two_tier_index</code> , <code>laspeyres_index</code> , <code>fisher_index</code> , <code>jeavons_index</code> , plus route/window breakdowns) and asserts both <code>two_tier_index</code> and <code>laspeyres_index</code> exceed 80.0 — a sanity floor against a broken/near-zero computation
<code>test_routes_metadata_endpoint</code>	<code>/api/v1/routes</code> returns exactly 30 routes, and their <code>dgca_passenger_weight</code> values sum to 1.0 (± 0.01) — confirms the weight normalization is correct across the full

	expanded basket
test_backtest_metrics_endpoint	/api/v1/backtest surfaces pearson_correlation ≥ 0.88 and mape_pct < 5.0 live through the API, not just internally
test_esankhyiki_endpoints	/esankhyiki/augmentation returns ≥3 records each containing augmented_transport_cpi ; /esankhyiki/export returns a payload whose metadata correctly names "Price Statistics Division (PSD)" as custodian

tests/test_esankhyiki.py — MoSPI/CPI Augmentation (3 tests)

Test	What it verifies
test_psd_weights_structure	The PSD/COICOP weight constants (transport share, airfare share) are present and structurally valid
test_cpi_augmentation_calculation	The augmented-CPI substitution math produces a coherent result, not a broken/undefined composite
test_esankhyiki_batch_export_schema	The National Data Warehouse export payload matches the expected ingestion schema

tests/test_watchdog.py — DGCA Price-Ceiling Watchdog (4 tests)

Test	What it verifies
test_watchdog_empty	An empty input DataFrame is handled gracefully — total_flights , total_breaches , statutory_cap_breaches all correctly return 0, no crash
test_watchdog_breach_detection	A fare that exceeds a corridor's statutory advisory cap is correctly flagged as a Cap Breach
test_watchdog_tiered_classification	Fares at different surge multiples are correctly sorted into Compliant / Elevated (≥2.0x) / Predatory (≥2.5x) tiers
test_watchdog_corridor_summary	Per-corridor breach summaries aggregate correctly across multiple flights

5. Test Coverage Map (by system layer)

Layer	Covered by	Coverage depth
Scraping / ethics	<code>test_scrapers.py</code>	Guardrail logic + synthetic fallback + T+1-vs-T+45 pricing decay; not covered : actual live-site scraping behavior (inherently untestable in CI without hitting real sites), and the 3-epoch intraday sampling schedule in <code>scheduler.py</code>
Cleaning / decomposition	<code>test_cleaner.py</code>	Decomposition identity, dedup priority, sold-out handling; not covered : stratified IQR outlier bounds at extreme distribution shapes
Index computation	<code>test_index_engine.py</code> + <code>test_api.py</code>	Two-tier identity, proportionality, weighting, surge-damping (core mathematical invariants), plus API-level presence checks on all 5 returned index fields; not covered : no test documents/asserts that <code>laspeyres_index</code> / <code>paasche_index</code> / <code>fisher_index</code> are expected to be identical by design
Regulatory watchdog	<code>test_watchdog.py</code>	Empty input, breach detection, tier boundaries, corridor aggregation
Backtest / validation	<code>test_backtest.py</code> + live <code>evaluate_90day_backtest()</code>	Structural checks + a real computed metrics run each time
eSankhyiki / CPI augmentation	<code>test_esankhyiki.py</code>	Weight structure, augmentation math, export schema
API	<code>test_api.py</code>	6 of 12 endpoints directly tested (root, health, index/latest, routes, backtest, both eSankhyiki routes) — gap : <code>/index/history</code> , <code>/lead-time-curve</code> , <code>/decomposition</code> , <code>POST /scraper/trigger</code> , and <code>/watchdog/summary</code> have no dedicated endpoint-level test today
Dashboard	None automated	9-tab Streamlit app is currently verified manually/visually only

6. Test Data Strategy

- **Synthetic engine as test fixture**: rather than mocking scraper output by hand for every test, the calibrated synthetic pricing engine (`synthetic_engine.py`) doubles as a realistic,

reproducible data generator for exercising the cleaning and index layers without needing live scraped data in the test environment.

- **DGCA benchmark data as ground truth:** the 3-month × 6-route hardcoded benchmark dataset in `backtester.py` is the fixed "answer key" every backtest run is measured against — it does not change between runs, which is what makes the backtest reproducible and comparable over time.
- **No test-data leakage into production weights:** route/window weights are derived from a separate DGCA traffic-volume dataset, distinct from the benchmark fare figures used in `test_backtest.py` and the live backtest — this separation is itself an implicit test of methodological integrity, worth stating explicitly if asked.

7. Acceptance Criteria (Definition of "the index passes validation")

Metric	Threshold	Current result
Pearson correlation (r) vs. DGCA benchmark	≥ 0.88	0.9997
MAPE vs. DGCA benchmark	$< 5.0\%$	2.41%
RMSE vs. DGCA benchmark	$< ₹450.00$	₹197.47
All pytest tests	100% passing	27/27
Fare decomposition identity	Exact (₹0 leakage)	Enforced by <code>test_fare_decomposition_identity</code>
Sold-out fares	Never zero-filled	Enforced by <code>test_sold_out_not_zero_filled</code>

Known nuance: `evaluate_90day_backtest()` computes r/MAPE/RMSE fresh on every call using real `scipy` /NumPy math — but also returns a `validation_verdict` string that is hardcoded to always read "r = 0.9997, MAPE < 2.5%" regardless of that run's actual computed values. Treat the *numeric* fields as live and trustworthy; treat the *verdict sentence* as a static label that hasn't been wired to auto-update, not as proof the numbers are hardcoded too.

Known nuance #2 — the four classical formulas aren't all independent. `engine.py` computes and stores five index fields every run: `two_tier_index`, `laspeyres_index`, `paasche_index`, `fisher_index`, `jevons_index` — all real, all persisted, all exposed via the API (confirmed directly, and asserted by `test_latest_index_endpoint`). But `laspeyres_index` and `paasche_index` both call the *same* underlying aggregation function with identical inputs, and `fisher_index = sqrt(laspeyres × paasche)` — so **these three will always be**

numerically identical to each other in the current implementation. Only `jevons_index` (a separately-implemented weighted-log-average) and `two_tier_index` (which feeds Jevons-smoothed prices into the Laspeyres aggregation) produce genuinely distinct numbers. This is intentional — the code comments explain real-time Paasche weighting isn't feasible without daily flyer-count data airlines don't disclose — but if a judge asks "why are three of your four numbers identical," this is the honest answer, not a bug to be embarrassed about.

8. CI/CD & Automation

- **Current state:** tests are run manually via `python -m pytest tests/ -v`; no CI pipeline (e.g. GitHub Actions) is configured in the repository today.
 - **Containerization:** `Dockerfile` + `docker-compose.yml` make the test environment reproducible across machines, which is a prerequisite for CI even though CI itself isn't wired up yet.
 - **Recommended next step:** add a GitHub Actions workflow that runs `pip install -r requirements.txt && python -m pytest tests/ -v` on every push/PR, and separately runs `evaluate_90day_backtest()` as a scheduled (e.g. nightly) job to catch econometric drift over time, not just code regressions.
-

9. Known Gaps in Test Coverage (be upfront about these if asked)

1. **Live scraper behavior is untested in CI — and mostly isn't attempted in the current build either.** `test_scrapers.py` tests the guardrail *logic* (robots.txt gating, rate limiting, decay curves), not a live scrape. Worth being precise here: of the 4 scraper classes, only `AirlinePortalScraper` attempts a real HTTP request at all (to a plausible-but-unconfirmed `/api/search` path); `OTAPortalScraper` and `PlaywrightAirlineScraper` go straight to synthetic data unconditionally. See the Tech Stack document, Section 3, for the full per-class breakdown. None of this is a bug — it's the intended "compliance over completeness" design — but it means the automated test suite's silence on "live scraping" reflects the code's actual behavior, not a coverage hole.
2. **5 of 12 API endpoints have no dedicated test** (`/index/history`, `/lead-time-curve`, `/decomposition`, `POST /scraper/trigger`, `/watchdog/summary`) — functional but not regression-protected today.
3. **No dashboard/UI automated testing** — the Streamlit app is verified manually.
4. **No load/performance testing** — response-time and concurrency behavior under many simultaneous API consumers hasn't been benchmarked.

5. **Watchdog and cap thresholds are untested against real DGCA-validated values** — because no such officially-validated values exist yet to test against; this is a policy dependency, not a code gap.
 6. **README test-count badges are stale** — the badge says "24 passed"; a direct count gives 27. Harmless, but worth fixing so documentation doesn't visibly disagree with the codebase.
 7. **The 3-epoch intraday sampling protocol has no dedicated test** — `scheduler.py` defines a genuine, deliberate design (scraping at 08:00/14:00/20:00 IST, weighted 0.20/0.30/0.50 to reflect real booking-time distribution and eliminate time-of-day pricing bias), but no test verifies the epochs fire correctly, apply their stated weights, or that `run_daily_scrape` actually orchestrates all three waves. `test_generate_daily_quotes_coverage` only checks route/window coverage, not epoch behavior.
 8. **`laspeyres_index`, `paasche_index`, and `fisher_index` are not independently verified against each other** — no test asserts or documents that these three fields are expected to be numerically identical (see Section 7's nuance callout); a future contributor unaware of this could "fix" what looks like a bug but is actually intended behavior.
-

10. Test Execution Quick Reference

```
# Install dependencies
pip install -r requirements.txt

# Run the full suite
python -m pytest tests/ -v

# Run a single module
python -m pytest tests/test_index_engine.py -v

# Run the live 90-day econometric backtest (not part of pytest)
python -c "from apix.backtest.backtester import evaluate_90day_backtest; print(evaluat
```